

BURP SUITE

ACADEMY / RAPPORT D'INTERVENTION SQLi

Rapport de Laboratoire 2

Attaque SQL UNION et recherche d'une colonne textuelle

Identification du canal textuel avec b6jsPw

Étudiant	Gana Faye
Formation	Sécurité des applications web
Plateforme	PortSwigger Web Security Academy
Laboratoire	SQL injection UNION attack, finding a column containing text
Module	Injections SQL
Niveau	Practitioner
Fenêtre d'intervention	13:19 à 13:41
Date	28 juillet 2026
Statut	Résolu
Niveau de risque	Élevé

Table des matières

1	Introduction et contexte du laboratoire	2
1.1	Présentation de l'exercice	2
1.2	Objectif technique	2
1.3	Périmètre et résultat attendu	2
2	Phase de reconnaissance et interception	3
2.1	Étape 1 – Ouverture et accès au laboratoire (13 :19)	3
2.2	Étape 2 – Capture de la requête HTTP (13 :23)	3
2.3	Étape 3 – Transfert vers Burp Repeater (13 :24)	4
3	Phase d'exploitation et test des types de données	5
3.1	Étape 4 – Rappel de la structure à trois colonnes (13 :28)	5
3.2	Étapes 5 et 6 – Test de la première colonne (13 :30–13 :31)	6
3.3	Étape 7 – Test de la deuxième colonne (13 :38)	8
3.4	Étape 8 – Validation de l'affichage (13 :40)	9
3.5	Synthèse comparative des essais	10
4	Clôture et validation du laboratoire	11
4.1	Étape 9 – Validation officielle (13 :41)	11
4.2	Chronologie de l'intervention	12
4.3	Portée du résultat	12
5	Recommandations de sécurité	12
5.1	Utiliser des requêtes préparées	12
5.2	Appliquer un typage fort	12
5.3	Limiter les privilèges SQL	13
5.4	Maîtriser les erreurs	13
5.5	Tests de non-régression	13
5.6	Conclusion	13

1 Introduction et contexte du laboratoire

1.1 Présentation de l'exercice

Le laboratoire *SQL injection UNION attack, finding a column containing text* poursuit l'étude de l'injection SQL présente dans le filtre de catégories de la boutique PortSwigger. La structure de la requête sous-jacente est connue : elle renvoie trois colonnes. L'étape suivante consiste à déterminer laquelle de ces positions accepte une chaîne de caractères et restitue cette valeur dans le rendu HTML.

Cette information est indispensable pour exploiter une attaque UNION orientée vers la récupération de données textuelles. Une colonne compatible mais non affichée ne permettrait pas d'observer directement les résultats ; une colonne affichée mais incompatible provoquerait une erreur de type. Le laboratoire demande donc de confirmer simultanément la compatibilité textuelle et la visibilité de la valeur injectée.

1.2 Objectif technique

La méthode consiste à remplacer successivement chaque valeur NULL par une chaîne aléatoire unique :

```
' UNION SELECT 'b6jsPw',NULL,NULL--  
' UNION SELECT NULL,'b6jsPw',NULL--  
' UNION SELECT NULL,NULL,'b6jsPw'--
```

Le marqueur `b6jsPw` est suffisamment distinctif pour être retrouvé sans ambiguïté dans la réponse. Dans les captures fournies, la première position provoque une erreur 500, tandis que la deuxième position retourne une réponse 200 OK et affiche le marqueur dans la page.

Pourquoi utiliser une chaîne aléatoire unique ?

Un marqueur prévisible comme `test` pourrait déjà apparaître dans le HTML, un script ou une donnée du catalogue. La chaîne `b6jsPw` permet d'attribuer avec confiance l'apparition du texte à la requête UNION. Elle constitue une preuve claire que la valeur injectée a traversé le SGBD puis le mécanisme de rendu de l'application.

1.3 Périmètre et résultat attendu

Élément	Valeur
Cible	PortSwigger Web Security Academy
Point d'entrée	paramètre <code>category</code> de <code>/filter</code>
Structure connue	trois colonnes
Marqueur textuel	<code>b6jsPw</code>
Signal d'incompatibilité	réponse 500 Internal Server Error
Signal de compatibilité	réponse 200 OK
Preuve décisive	affichage de <code>b6jsPw</code> dans la page
Résultat	deuxième colonne compatible avec du texte et visible

Table 1 – Synthèse de l'objectif du laboratoire.

Périmètre autorisé

Les manipulations sont réalisées uniquement dans le laboratoire PortSwigger prévu pour cet apprentissage. Les mêmes charges ne doivent pas être testées sur un système tiers sans autorisation explicite.

2 Phase de reconnaissance et interception

2.1 Étape 1 – Ouverture et accès au laboratoire (13 :19)

Le laboratoire est initialisé dans le navigateur. La page confirme le scénario *finding a column containing text* et présente l'interface e-commerce habituelle. Le statut initial est *Not solved*, ce qui établit l'état de référence avant toute injection.

La navigation dans les catégories permet de déclencher la requête de filtrage vulnérable. À ce stade, l'objectif n'est pas encore d'injecter une charge, mais d'identifier le flux HTTP exact qui transporte la valeur contrôlée.

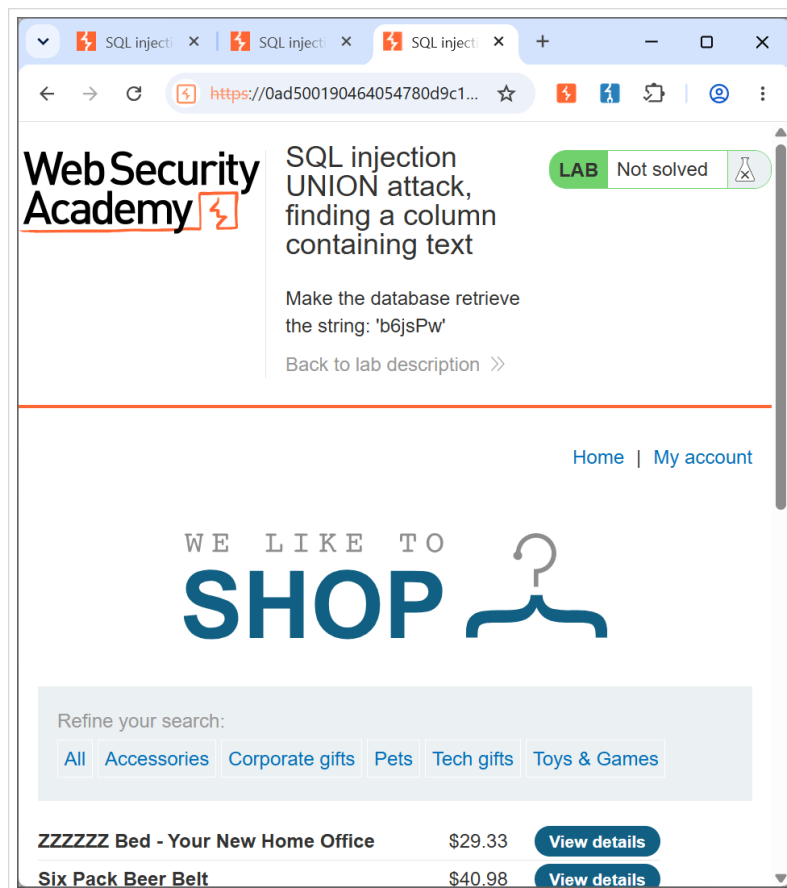


Figure 1 – Ouverture du laboratoire et état initial Not solved à 13 :19.

Référence de preuve : Labo_2_injection_sql/Capture_1_ouverturedulabo.png.

2.2 Étape 2 – Capture de la requête HTTP (13 :23)

L'historique HTTP de Burp Suite est consulté après la sélection d'une catégorie. La requête `GET /filter?category=...` est repérée parmi les échanges. Le paramètre `category` constitue le point

d'injection puisqu'il influence directement la sélection des produits.

La conservation de la requête légitime fournit une référence fonctionnelle : elle permet de comparer chaque réponse modifiée au comportement normal de l'application.

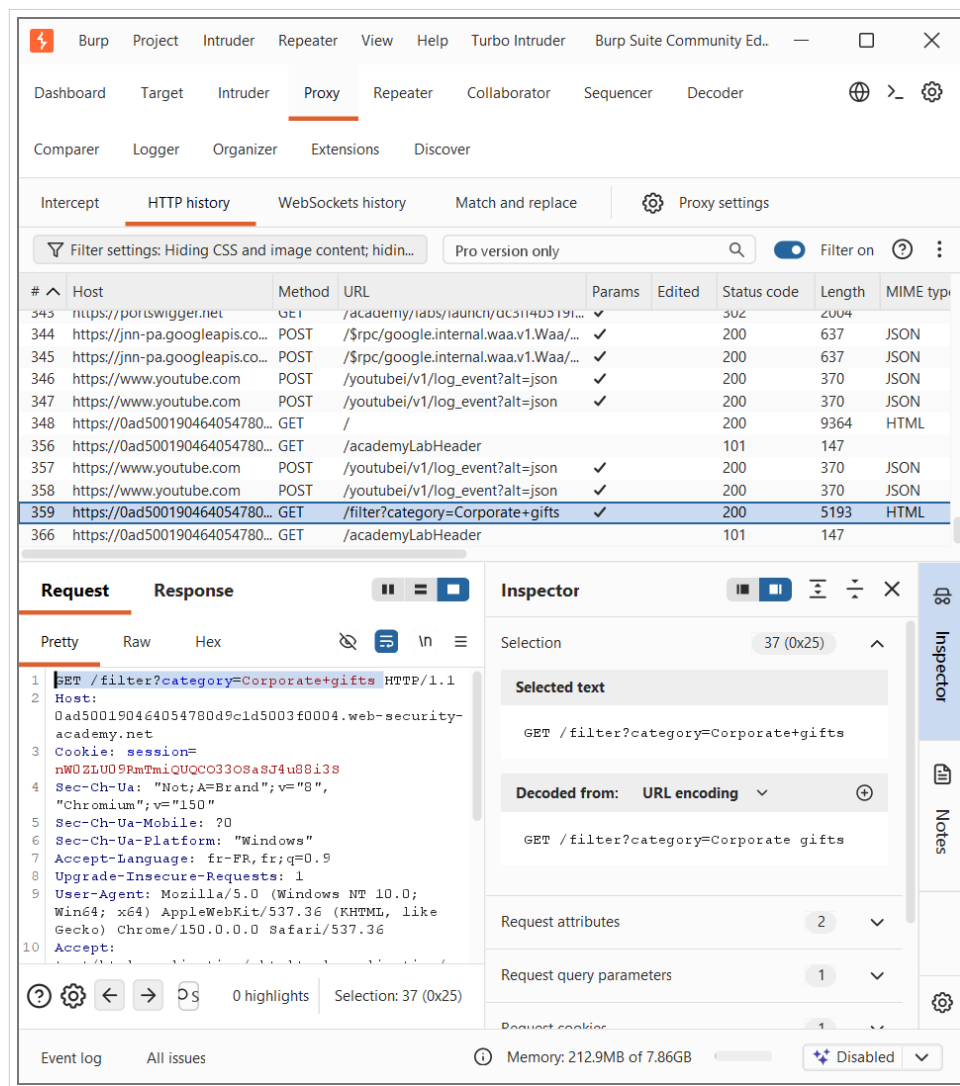


Figure 2 – Requête de filtrage identifiée dans l'historique HTTP de Burp Suite à 13 :23.

Référence de preuve : Labo_2_injection_sql/Capture_2_requête_capturée.png.

2.3 Étape 3 – Transfert vers Burp Repeater (13 :24)

La requête ciblée est envoyée vers Burp Repeater. Cet outil permet de déplacer le marqueur `b6jsPw` d'une colonne à l'autre tout en conservant le même endpoint, les mêmes en-têtes et le même contexte de session.

La démarche reste contrôlée : une seule position est modifiée entre deux essais. Toute différence de statut ou de contenu peut ainsi être reliée au type de la colonne testée.

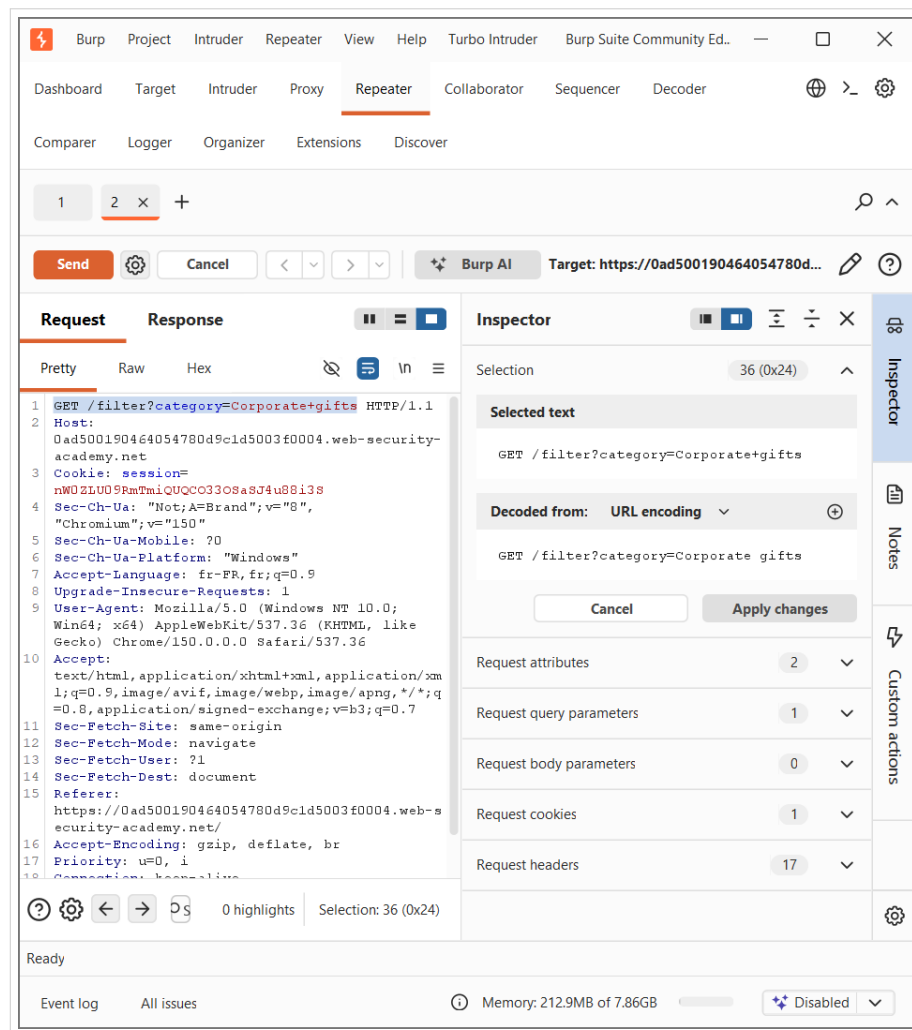


Figure 3 – Requête transférée dans Burp Repeater pour les tests itératifs à 13 :24.

Référence de preuve : Labo_2_injection_sql/Capture_3_envoi_repeteur.png.

Comportement sécurisé attendu

Le paramètre `category` devrait être lié comme une valeur dans une requête préparée. Le client ne devrait jamais pouvoir ajouter une projection UNION, déplacer des expressions entre les colonnes ou modifier le type des résultats.

3 Phase d'exploitation et test des types de données

3.1 Étape 4 – Rappel de la structure à trois colonnes (13 :28)

Avant de tester les types, une charge composée de trois valeurs NULL est rejouée :

```
' + UNION + SELECT + NULL, NULL, NULL --
```

Après décodage URL, la projection correspond à :

```
' UNION SELECT NULL, NULL, NULL --
```

La requête est acceptée, ce qui confirme que le nombre de colonnes est correct. Cette vérification

préalable est importante : si la structure était incorrecte, une erreur ultérieure ne permettrait pas de distinguer un problème de nombre de colonnes d'une incompatibilité de type.

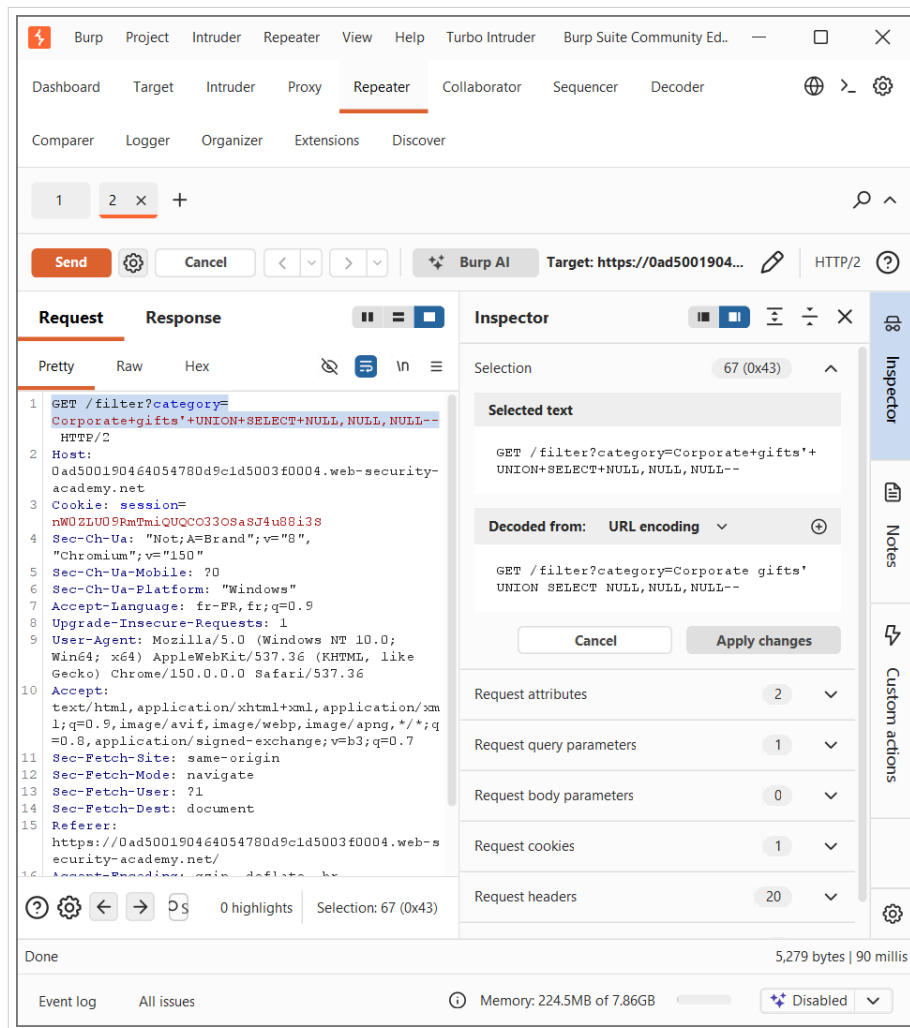


Figure 4 – Validation préalable de la structure à trois colonnes avec trois valeurs NULL à 13 :28.

Référence de preuve : Labo_2_injection_sql/Capture_4_1ere_injection_null.png.

Rôle de NULL

La valeur NULL est utilisée comme valeur neutre, généralement compatible avec de nombreux types SQL. Elle permet d'établir la forme de la projection avant d'introduire une chaîne, qui pourrait être incompatible avec une colonne numérique, booléenne ou d'un autre type strict.

3.2 Étapes 5 et 6 – Test de la première colonne (13 :30–13 :31)

Le premier NULL est remplacé par le marqueur textuel :

```
' +UNION+SELECT+'b6jsPw',NULL,NULL--
```

La forme SQL correspondante est :

```
' UNION SELECT 'b6jsPw',NULL,NULL--
```

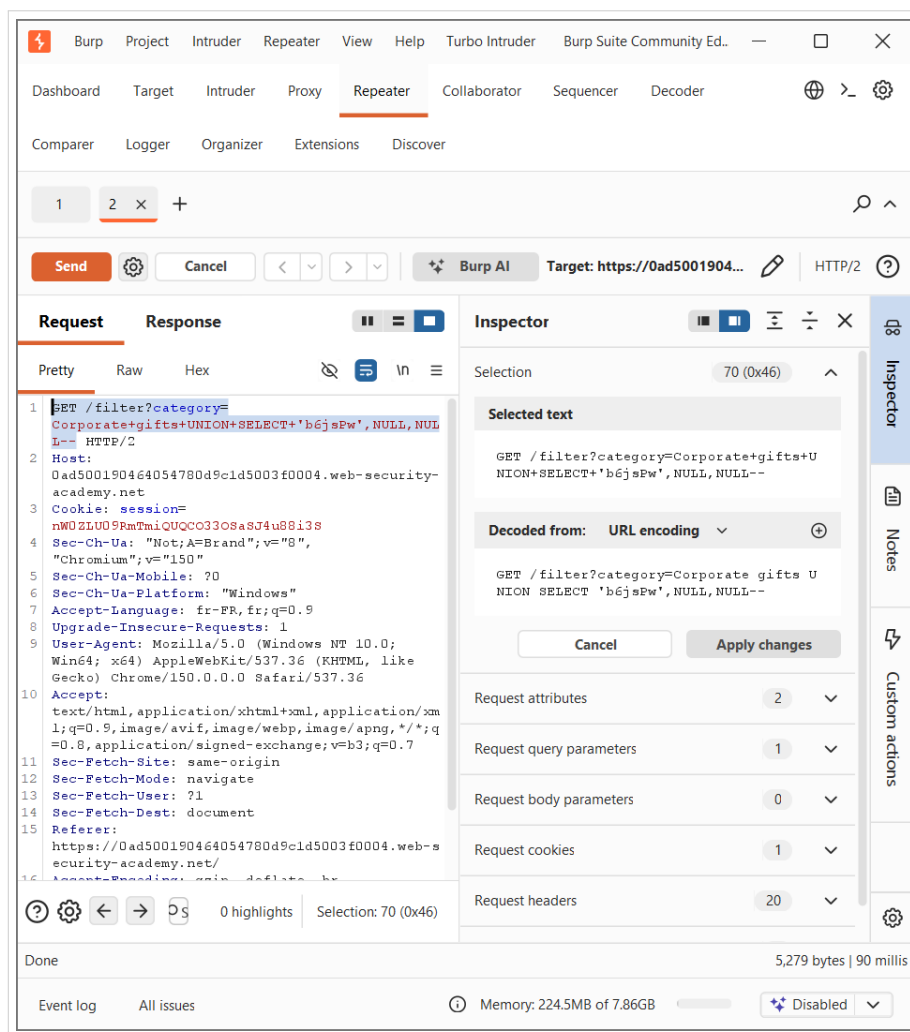


Figure 5 – Insertion de la chaîne `b6jsPw` dans la première colonne à 13 :30.

Référence de preuve : Labo_2_injection_sql/Capture_5_2eme_injection_avec_la_chaine.png.

Le serveur retourne ensuite 500 Internal Server Error. La structure comporte toujours trois colonnes ; la différence provient donc du remplacement d'un NULL par une chaîne. Dans ce contexte contrôlé, le résultat indique que la première position n'accepte pas la valeur textuelle utilisée, probablement parce que la colonne originale attend un type incompatible.

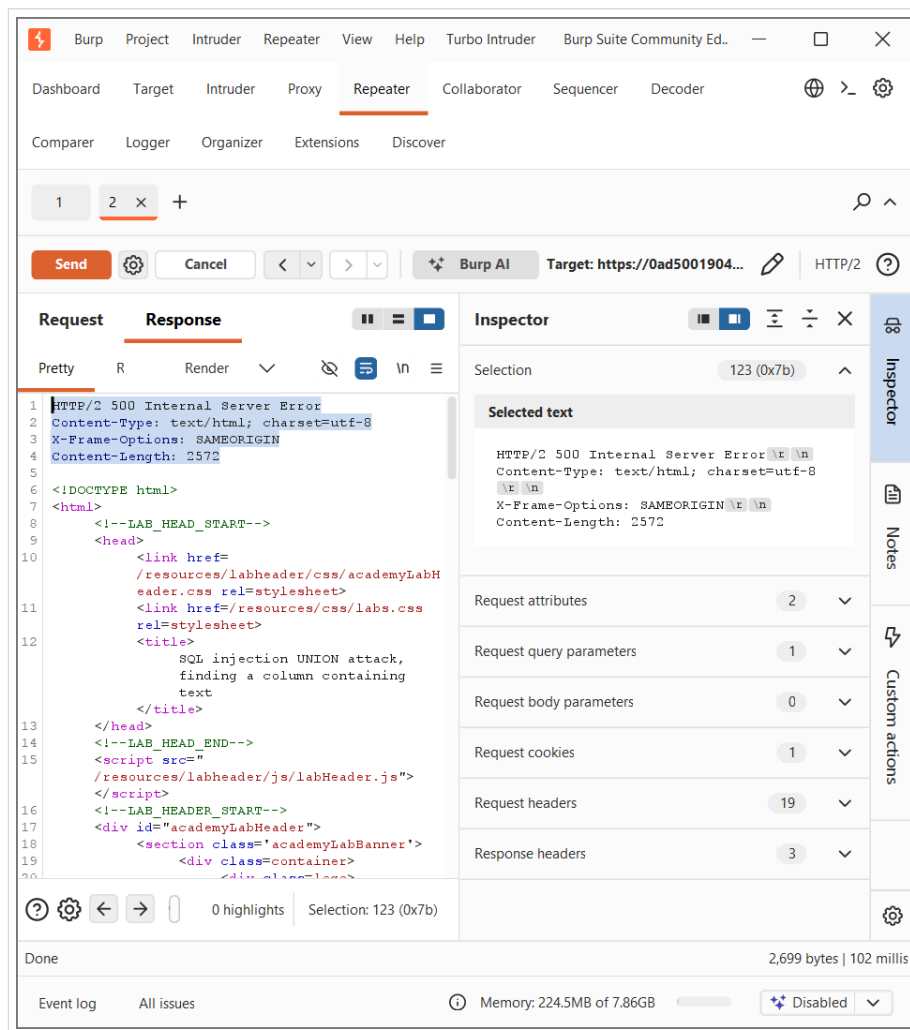


Figure 6 – Erreur 500 après l'insertion du texte dans la première colonne à 13 :31.

Référence de preuve :

Labo_2_injection_sql/Capture_6_2eme_injection_avec_la_chaine_erreur_500.png.

Interprétation de l'erreur 500

Une réponse 500 isolée ne révèle pas automatiquement le type d'une colonne. Ici, l'hypothèse repose sur une comparaison précise : trois NULL fonctionnent, puis la même projection échoue lorsque seule la première valeur devient textuelle. Le changement contrôlé rend l'incompatibilité de type très probable.

3.3 Étape 7 – Test de la deuxième colonne (13 :38)

La chaîne est déplacée vers la deuxième position, tandis que les première et troisième colonnes reçoivent à nouveau NULL :

```
'+UNION+SELECT+NULL,'b6jsPw',NULL--
```

```
' UNION SELECT NULL,'b6jsPw',NULL--
```

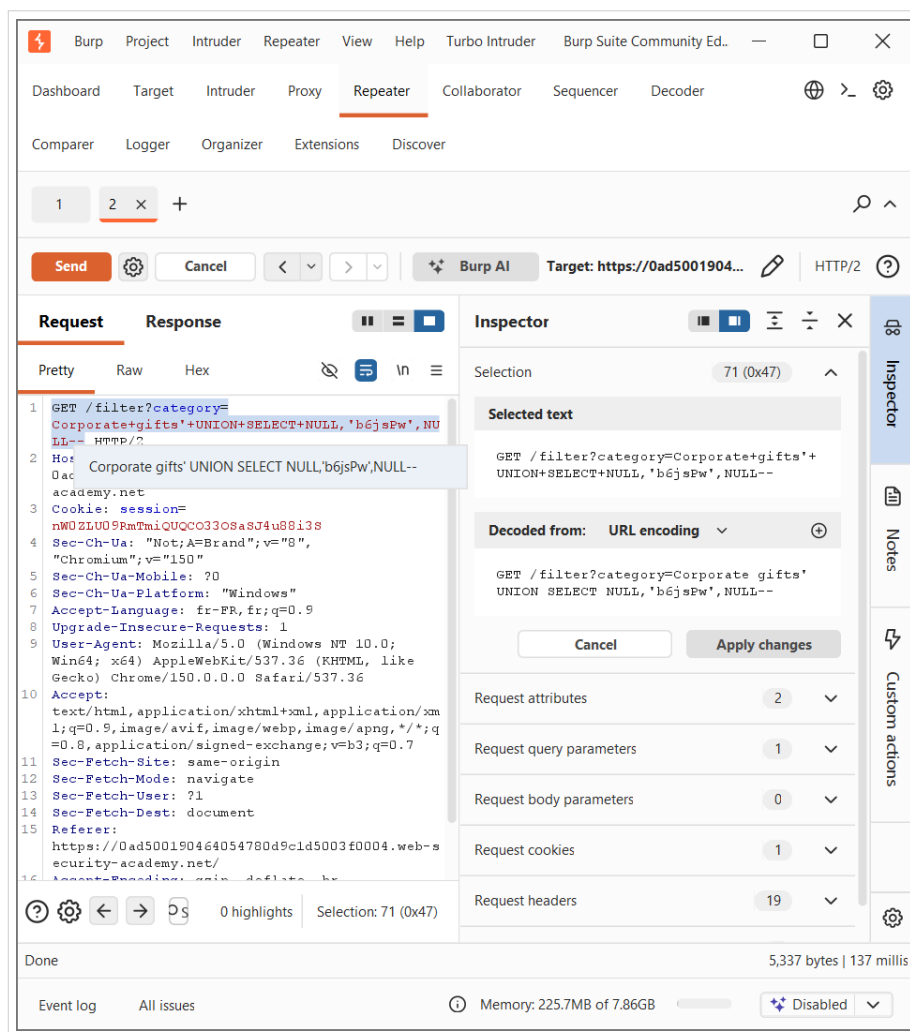


Figure 7 – Déplacement du marqueur `b6jsPw` vers la deuxième colonne à 13 :38.

Référence de preuve : Labo_2_injection_sql/Capture_7_injection_2Àlmeccolonne.png.

3.4 Étape 8 – Validation de l’affichage (13 :40)

La réponse associée au second emplacement est valide. Le laboratoire affiche directement la chaîne `b6jsPw` dans le rendu de la page. Deux propriétés sont donc confirmées :

1. la deuxième colonne accepte une valeur de type texte ;
2. cette colonne est utilisée par le template HTML et son contenu est visible.

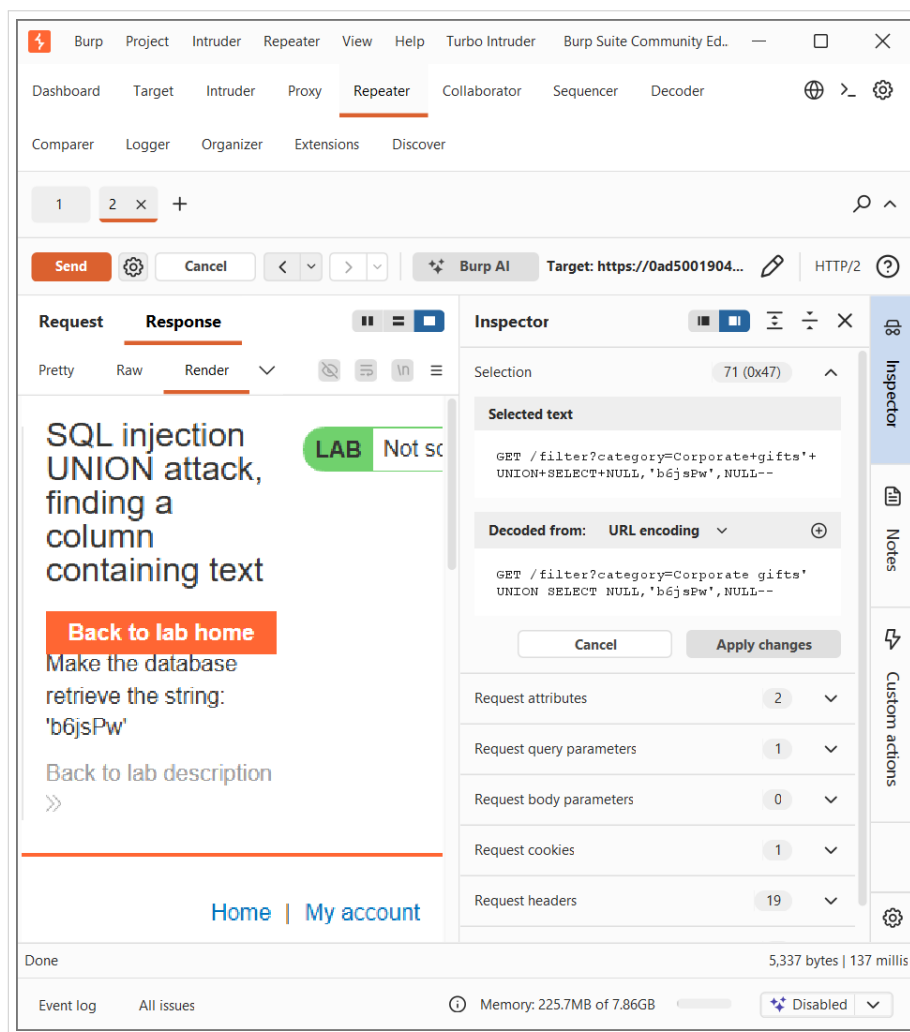


Figure 8 – Affichage réussi de la chaîne `b6jsPw` dans la page à 13 :40.

Référence de preuve : Labo_2_injection_sql/Capture_8_injection_reussis.png.

Pourquoi le code 200 ne suffit-il pas seul ?

Une réponse 200 OK montre que la requête et le rendu n'ont pas provoqué d'erreur, mais elle ne prouve pas nécessairement que la valeur est visible. L'apparition exacte de `b6jsPw` dans la page constitue la preuve décisive que la deuxième colonne est à la fois compatible et restituée au client.

3.5 Synthèse comparative des essais

Essai	Projection UNION	Réponse	Interprétation
1	NULL,NULL,NULL	succès	structure à trois colonnes confirmée
2	'b6jsPw',NULL,NULL	500	première position incompatible avec le texte testé
3	NULL,'b6jsPw',NULL	200	deuxième position textuelle et visible

Table 2 – Comparaison des tests de compatibilité textuelle.

Résultat technique confirmé

La **deuxième colonne** accepte des chaînes et affiche directement leur contenu.

4 Clôture et validation du laboratoire

4.1 Étape 9 – Validation officielle (13 :41)

Après l’affichage du marqueur, la plateforme Web Security Academy indique que le laboratoire est résolu. Le statut *Solved* et le bandeau de félicitations valident officiellement la démarche.

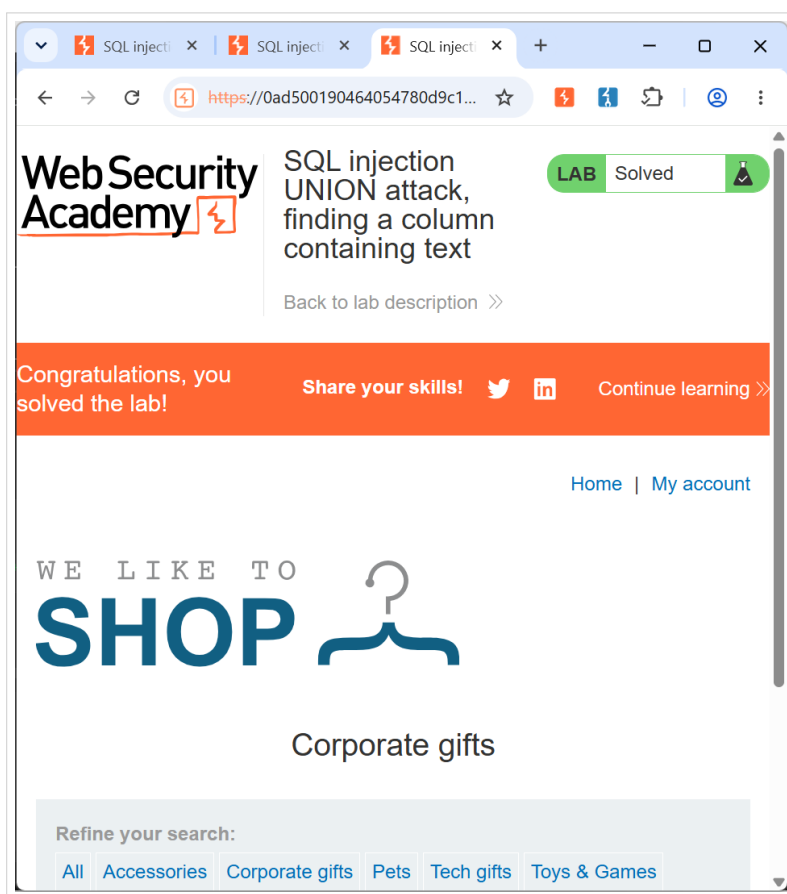


Figure 9 – Validation officielle du laboratoire et statut *Solved* à 13 :41.

Référence de preuve : Labo_2_injection_sql/Capture_9_labo_reussis.png.

4.2 Chronologie de l'intervention

Heure	Phase	Action
13 :19	Reconnaissance	ouverture du laboratoire
13 :23	Interception	capture de la requête de filtrage
13 :24	Préparation	transfert vers Burp Repeater
13 :28	Structure	validation des trois colonnes avec NULL
13 :30–13 :31	Type	première colonne testée, réponse 500
13 :38	Type	déplacement du marqueur en deuxième position
13 :40	Validation	réponse 200 et affichage de b6jsPw
13 :41	Clôture	laboratoire officiellement résolu

Table 3 – Chronologie synthétique du laboratoire 2.

4.3 Portée du résultat

La découverte d'une colonne textuelle visible permettrait, dans un scénario d'exploitation plus avancé, d'y placer des informations provenant d'autres tables, par exemple des noms d'utilisateurs ou des métadonnées du schéma. Le laboratoire s'arrête volontairement à l'identification du canal d'affichage.

Le test de la troisième colonne n'était plus nécessaire pour résoudre l'exercice. L'absence de test ne permet toutefois pas d'affirmer qu'elle est incompatible avec le texte ; seule la deuxième position est positivement confirmée comme textuelle et visible par les preuves disponibles.

5 Recommandations de sécurité

5.1 Utiliser des requêtes préparées

La correction principale consiste à séparer la structure de la requête et la valeur de catégorie :

```
SELECT id, name, price
FROM products
WHERE category = ? AND released = ?
```

La valeur reçue est liée au premier emplacement avec son type. Elle ne peut plus introduire un opérateur UNION, ajouter de nouvelles colonnes ou commenter la suite de l'instruction.

5.2 Appliquer un typage fort

Les valeurs doivent être validées selon leur rôle métier :

- la catégorie doit appartenir à une liste de catégories autorisées ;
- les identifiants numériques doivent être convertis et bornés ;
- les clés de tri doivent être traduites par une liste blanche ;
- les types attendus doivent être imposés par les DTO et le pilote SQL.

Le typage fort réduit les entrées incohérentes, mais ne remplace jamais la paramétrisation.

5.3 Limiter les privilèges SQL

Le compte utilisé pour le catalogue doit uniquement pouvoir consulter les tables nécessaires. Il ne doit pas accéder aux secrets d'authentification, aux fonctions administratives ou aux tables sans rapport avec cette fonctionnalité.

5.4 Maîtriser les erreurs

Les exceptions SQL doivent être journalisées côté serveur avec un identifiant de corrélation, puis remplacées par un message générique côté client. Le masquage de l'erreur réduit la divulgation, mais la cause racine reste la construction non paramétrée de la requête.

5.5 Tests de non-régression

Après correction, les tests suivants doivent être automatisés :

1. vérifier le fonctionnement des catégories légitimes ;
2. rejouer la projection à trois NULL et confirmer qu'elle est traitée comme une valeur ordinaire ;
3. tester `b6jsPw` dans chacune des positions et vérifier l'absence de modification SQL ;
4. contrôler que les erreurs ne varient pas selon le type de charge ;
5. confirmer qu'aucun marqueur injecté n'apparaît via une requête UNION ;
6. vérifier les permissions minimales du compte de base.

Comportement sécurisé attendu

Après correction, la chaîne `'+UNION+SELECT+NULL,'b6jsPw',NULL-` doit être rejetée comme catégorie invalide ou traitée comme une valeur littérale inexistante. Elle ne doit jamais modifier la projection SQL ni afficher le marqueur.

5.6 Conclusion

Le laboratoire a permis d'identifier méthodiquement une colonne capable de transporter et d'afficher du texte. La projection à trois NULL a d'abord confirmé la structure. L'insertion de `b6jsPw` dans la première position a provoqué une erreur 500, révélant une incompatibilité avec le type attendu. Son déplacement dans la deuxième position a produit une réponse 200 OK et l'affichage exact du marqueur.

La deuxième colonne est donc confirmée comme compatible avec les chaînes et visible dans la réponse. Cette connaissance permettrait de poursuivre une attaque UNION vers la récupération de données textuelles. La remédiation durable repose sur les requêtes préparées, le typage fort, une validation métier, le moindre privilège et des tests de non-régression.

Bilan final

Le laboratoire est résolu. La séquence `3 NULL = structure valide, texte en colonne 1 = erreur 500, puis texte en colonne 2 = 200 + affichage` démontre que la deuxième colonne constitue le canal textuel utile.